FESOJAYE AYOMIDE OLUWAGBENGA VAPT REPORT DONE ON BWAPP

SCOPE

This Vulnerability Assessment and Penetration Testing (VAPT) exercise was conducted to evaluate the security posture of a deliberately vulnerable web application known as bWAPP (Buggy Web Application). The purpose of this assessment was to simulate common web-based attack techniques in order to identify security weaknesses that could potentially be exploited by malicious actors.

The assessment was performed within a controlled laboratory environment using Kali Linux as the primary penetration testing platform. The target application was deployed locally using Docker, ensuring that all testing activities were confined to an isolated system and did not impact any external or production systems. Web traffic between the browser and the application was monitored and manipulated using Burp Suite, which enabled detailed inspection and modification of HTTP requests and responses during the testing process.

The scope of this engagement focused specifically on identifying and exploiting common web application vulnerabilities that are frequently encountered in real-world systems. The vulnerabilities examined during this assessment include:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- Broken Authentication
- Insecure Direct Object Reference (IDOR)

Testing activities were limited to the functionality and components of the bWAPP application accessible through the web interface. Network infrastructure, operating system hardening, and external services outside the application environment were not included within the scope of this assessment.

The objective of the testing was to demonstrate how improper input validation, weak access controls, and insecure application design can be exploited by attackers to gain unauthorized access, manipulate application data, or execute unintended commands on the server. All findings documented in this report were identified through controlled testing techniques and are presented along with evidence of exploitation and recommended remediation measures.

EXECUTIVE SUMMARY

This report presents the results of a Vulnerability Assessment and Penetration Testing (VAPT) engagement conducted against the deliberately vulnerable web application bWAPP. The objective of this assessment was to identify common security weaknesses in web applications and demonstrate how these vulnerabilities can be exploited by an attacker to compromise system integrity, confidentiality, and availability.

Testing was performed in a controlled laboratory environment using Kali Linux as the primary penetration testing platform. The target application was deployed locally using Docker, and web traffic between the client and server was intercepted and analyzed using Burp Suite. These tools enabled detailed inspection of application behavior, manipulation of HTTP requests, and controlled exploitation of identified vulnerabilities.

During the course of the assessment, multiple security weaknesses were successfully identified and exploited. The testing focused on widely recognized categories of web application vulnerabilities aligned with industry standards such as the OWASP Top 10. The vulnerabilities discovered demonstrate how insufficient input validation, weak authentication mechanisms, and improper access control can expose critical components of a web application to malicious manipulation.

Five key vulnerabilities were successfully exploited during the assessment:

1. SQL Injection – allowing unauthorized interaction with the application's database and retrieval of unintended data.
2. Cross-Site Scripting (XSS) – enabling execution of arbitrary scripts within a user's browser session.
3. Command Injection – allowing system-level commands to be executed on the server through un-sanitized input.
4. Broken Authentication – exposing weaknesses in the application's authentication mechanisms that could allow unauthorized access.
5. Insecure Direct Object Reference (IDOR) – permitting manipulation of user-controlled parameters to access or modify resources belonging to other users.

The successful exploitation of these vulnerabilities highlights the significant risks that arise when secure coding practices, input validation, and proper authorization controls are not implemented. In a real-world environment, vulnerabilities of this nature could enable attackers to extract sensitive data, escalate privileges, manipulate user accounts, or execute commands on the underlying system.

This report documents the methodology used during testing, detailed technical findings for each vulnerability identified, supporting evidence of exploitation, and recommended remediation measures aimed at mitigating these risks. Implementing these recommendations would significantly strengthen the security posture of the application and reduce the likelihood of successful exploitation by malicious actors.

Overall, the results of this assessment demonstrate the critical importance of incorporating secure development practices, continuous vulnerability testing, and proactive security monitoring throughout the software development lifecycle.

TOOLS USED

The vulnerability assessment and penetration testing activities conducted during this project relied on a combination of specialized cybersecurity tools designed to analyze web application behavior, intercept network traffic, and exploit potential security weaknesses. These tools enabled controlled testing of the target application and provided the necessary capabilities to identify and validate security vulnerabilities.

1. **Kali Linux :**

Kali Linux served as the primary penetration testing environment for this assessment. It is a security-focused Linux distribution widely used by cybersecurity professionals and ethical hackers.

2. **Burp Suite:**

Burp Suite was used as the primary web application security testing tool. It acted as an intercepting proxy between the web browser and the target application, allowing HTTP requests and responses to be captured, inspected, and modified. This capability made it possible to analyze application behavior, manipulate input parameters, test for vulnerabilities such as SQL injection and authentication flaws, and validate the effectiveness of exploit attempts.

3. **Docker:**

Docker was used to deploy and manage the vulnerable web application within a containerized environment. Containerization allowed the application to run in an isolated and controlled testing environment, ensuring that all testing activities remained confined to a local system without affecting external infrastructure.

4. **Web Browser:**

A standard web browser was used to interact with the target application during the testing process. The browser allowed navigation through the web interface, submission of input data, and observation of application responses. When configured to route traffic through Burp Suite, it enabled full visibility into the communication between the client and the server.

5. **bWAPP** (Buggy Web Application):

The target of this assessment was the Buggy Web Application (bWAPP), a deliberately vulnerable web application designed for security training and penetration testing practice. bWAPP contains numerous intentionally embedded vulnerabilities that allow testers to safely practice exploiting common web security flaws such as SQL injection, Cross-Site Scripting, and authentication weaknesses.

Together, these tools provided a comprehensive environment for conducting systematic web application security testing and demonstrating the exploitation of common vulnerabilities in a controlled laboratory setting.

METHODOLOGY

The Vulnerability Assessment and Penetration Testing (VAPT) exercise followed a structured testing methodology designed to systematically identify, analyze, and exploit security weaknesses within the target web application bWAPP. The testing process was conducted using a combination of manual analysis and controlled exploitation techniques to simulate real-world attack scenarios. The methodology adopted during this assessment consisted of several sequential phases, each aimed at progressively identifying potential security flaws and validating their exploitability.

1. **Environment Setup**:

The first phase involved preparing the testing environment required to safely conduct the assessment. The vulnerable web application was deployed locally using Docker, ensuring that all testing activities were performed within an isolated laboratory environment. The penetration testing platform used for this assessment was Kali Linux, which provided the necessary tools and utilities required for web application security testing.

A web browser was configured to route all HTTP traffic through Burp Suite, allowing the interception and inspection of requests and responses exchanged between the client and the web application.

2. **Application Reconnaissance:**

During this phase, the structure and functionality of the application were explored in order to identify potential entry points for user input. Various components of the web application were examined, including login forms, search fields, parameterized URLs, and input fields where user-supplied data could be submitted to the server.

This step helped in identifying areas where improper input validation or weak security controls might exist.

3. **Traffic Interception and Analysis**:

Once potential input points were identified, HTTP requests generated by the application were intercepted using Burp Suite. This allowed detailed analysis of how user input was transmitted to the server and how the server responded to those inputs.

Parameters within intercepted requests were closely examined to determine whether they could be manipulated to alter the behavior of the application.

4. **Input Manipulation and Vulnerability**: Testing

After identifying parameters that accepted user input, controlled payloads were injected into these parameters to test for common web application vulnerabilities. Examples of testing techniques used include:

Injecting SQL syntax to identify potential SQL Injection vulnerabilities

Injecting HTML and JavaScript code to test for Cross-Site Scripting (XSS)

Modifying authentication parameters to evaluate authentication weaknesses

Manipulating object references to test for access control flaws

Submitting malicious input to determine whether server-side commands could be executed Each test was performed carefully to observe the server's behavior and determine whether the application improperly handled the injected input.

5. **Exploitation and Verification:**

When abnormal behavior or error messages were observed, further testing was conducted to confirm whether the vulnerability could be reliably exploited. Successful exploitation was validated by demonstrating the ability to manipulate application behavior, retrieve unintended data, execute commands, or bypass security mechanisms.

All successful exploit attempts were documented with screenshots showing the injected payload, the intercepted request, and the resulting server response.

6. **Documentation of findings:**

Once vulnerabilities were confirmed, detailed documentation was prepared describing each vulnerability, the methodology used to exploit it, the evidence of exploitation, and recommended remediation measures. This ensured that each finding could be clearly understood and reproduced for verification purposes.

This structured methodology ensured that the assessment was performed in a systematic and controlled manner, enabling accurate identification of security weaknesses and clear documentation of the associated risks.

VULNERABILITIES IDENTIFIED

The security assessment of bWAPP revealed several web application vulnerabilities resulting from weaknesses in authentication mechanisms, input validation, and session management.

The following sections describe the identified vulnerabilities, the methodology used to discover them, and recommended remediation measures

Identification and Authentication Failures Methodology:

Authentication mechanisms of the target application bWAPP were tested to evaluate the effectiveness of login protection mechanisms, session management practices, and the secure handling of authentication credentials.

Testing involved interacting with the login interface and authentication-related functionalities while intercepting HTTP requests and responses using Burp Suite. This allowed detailed inspection of authentication parameters, session identifiers, and server responses to determine whether proper security controls were implemented.

The assessment focused on identifying weaknesses such as brute force protection failures, insecure session handling, improper logout mechanisms, and transmission of credentials over insecure communication channels.

Vulnerabilities Identified

1. **Brute Force Password Attack Vulnerability**

The login mechanism did not implement any protection against automated password guessing attacks. Multiple login attempts using different password values were allowed without triggering any account lockout mechanism, rate limiting, or CAPTCHA challenge.

Testing involved submitting repeated authentication attempts with different passwords. The application continued to process these requests without restriction, indicating that an attacker could perform automated brute force attacks to discover valid credentials.

This weakness significantly increases the risk of unauthorized account compromise.

2. **Session Identifier Exposure in URL**

During testing, it was observed that the application exposed the session identifier directly within the URL parameters. Session identifiers should be securely stored within cookies rather than being transmitted through URLs.

Exposing session IDs in URLs increases the likelihood of session hijacking because the session token may be captured through browser history, server logs, or network monitoring tools.

3. **Insecure Login Form (Use of HTTP)**

Analysis of intercepted authentication requests revealed that login credentials were transmitted over an unencrypted HTTP connection rather than using HTTPS.

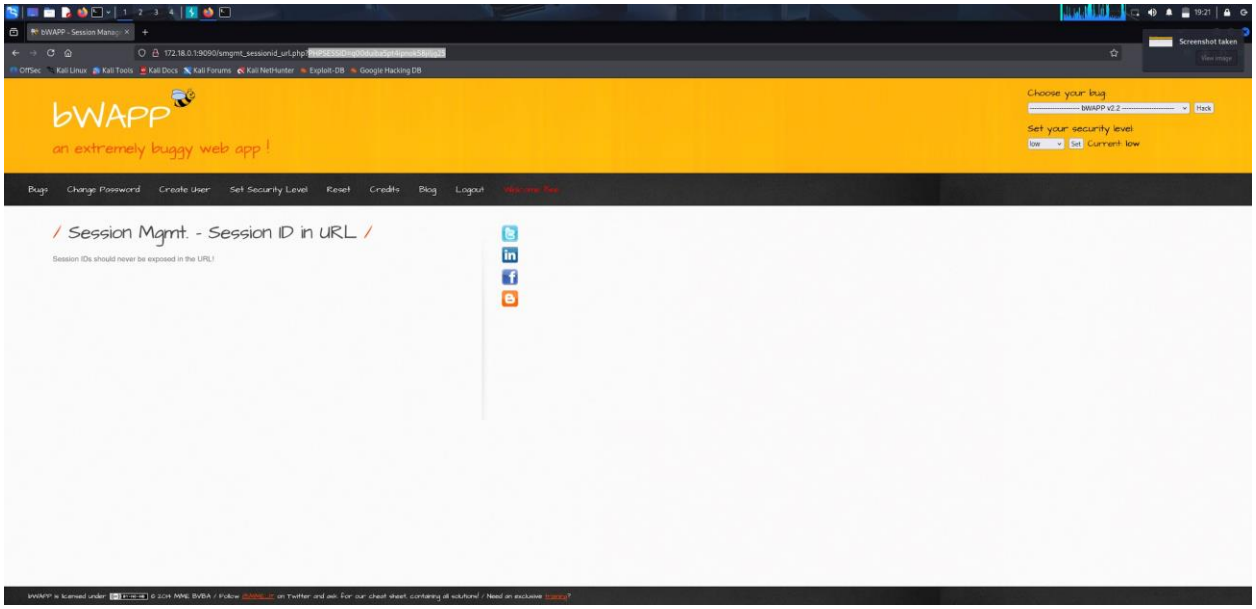
Because HTTP traffic is not encrypted, sensitive information such as usernames and passwords may be intercepted by attackers through network sniffing or man-in-the-middle attacks.

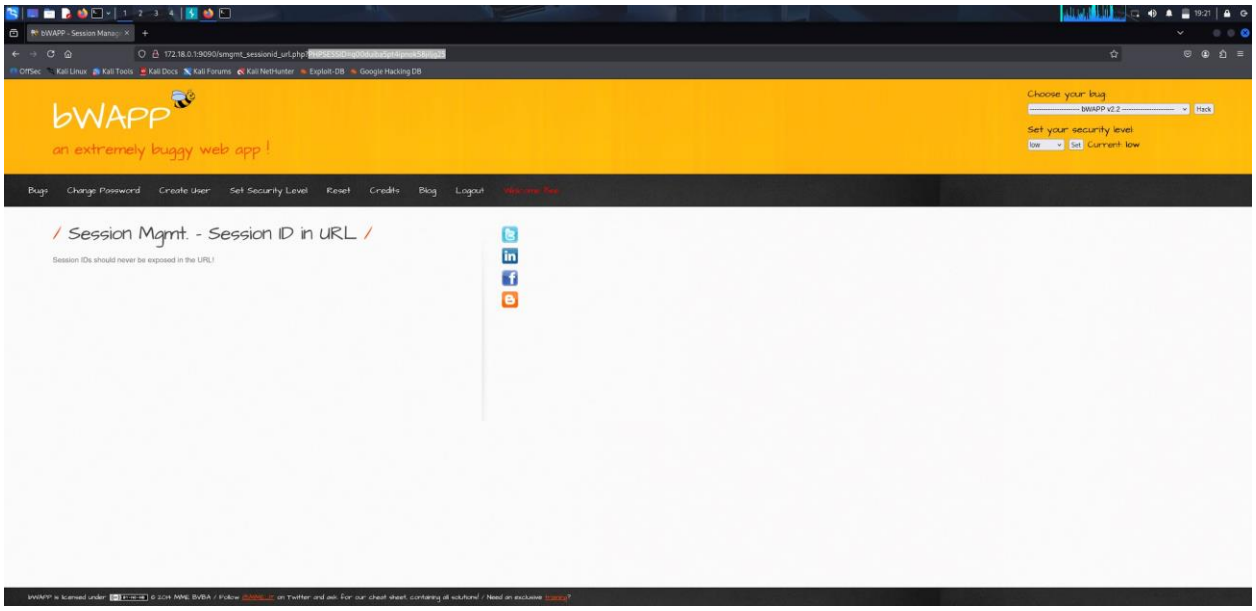
4.**Improper Session Termination (Logout Management Weakness)**

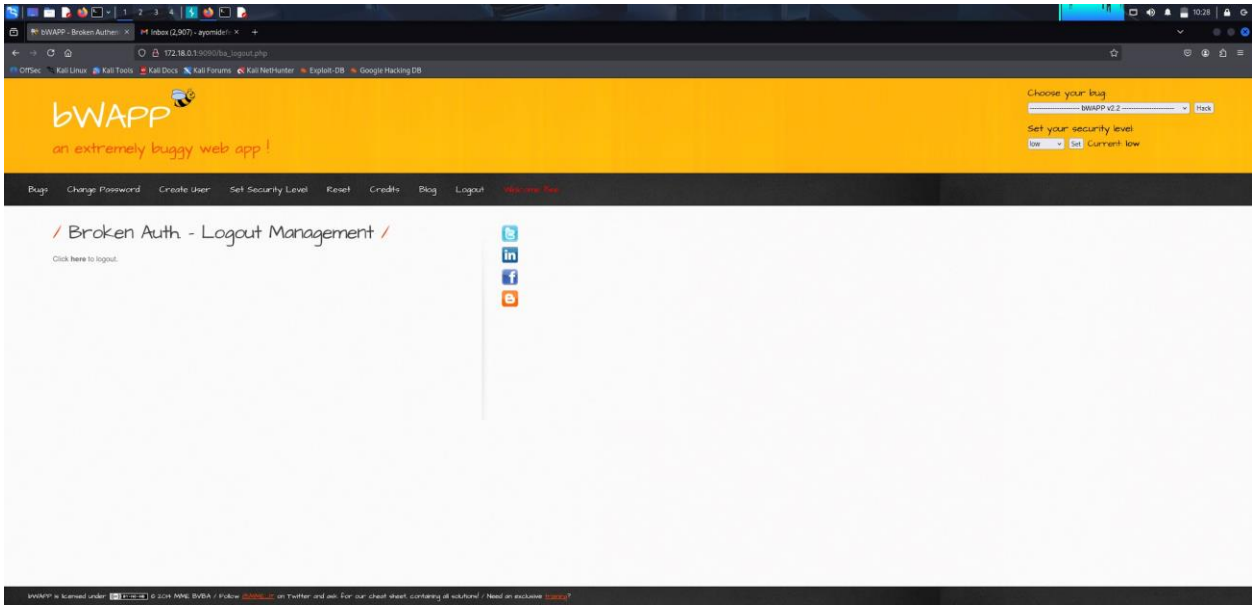
Testing of the logout functionality revealed that logging out of the application did not immediately invalidate the active session. The session cookie remained valid until another login attempt was performed.

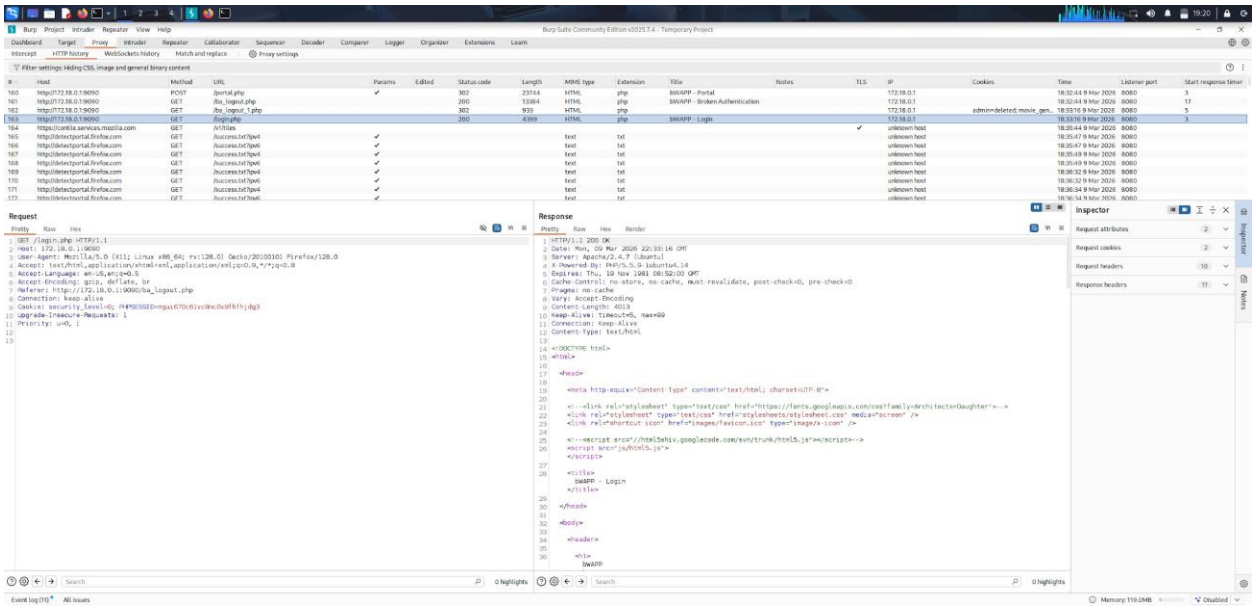
This behavior indicates improper session termination, as a valid session token should be invalidated immediately upon logout to prevent unauthorized reuse.

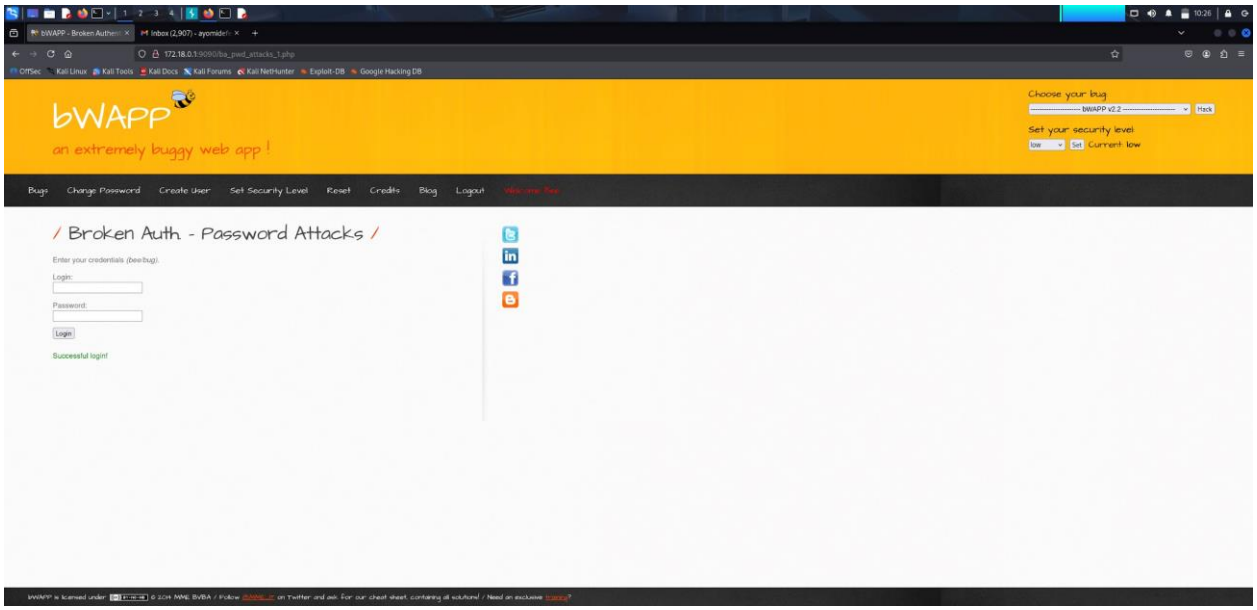
Screenshots of Exploitation

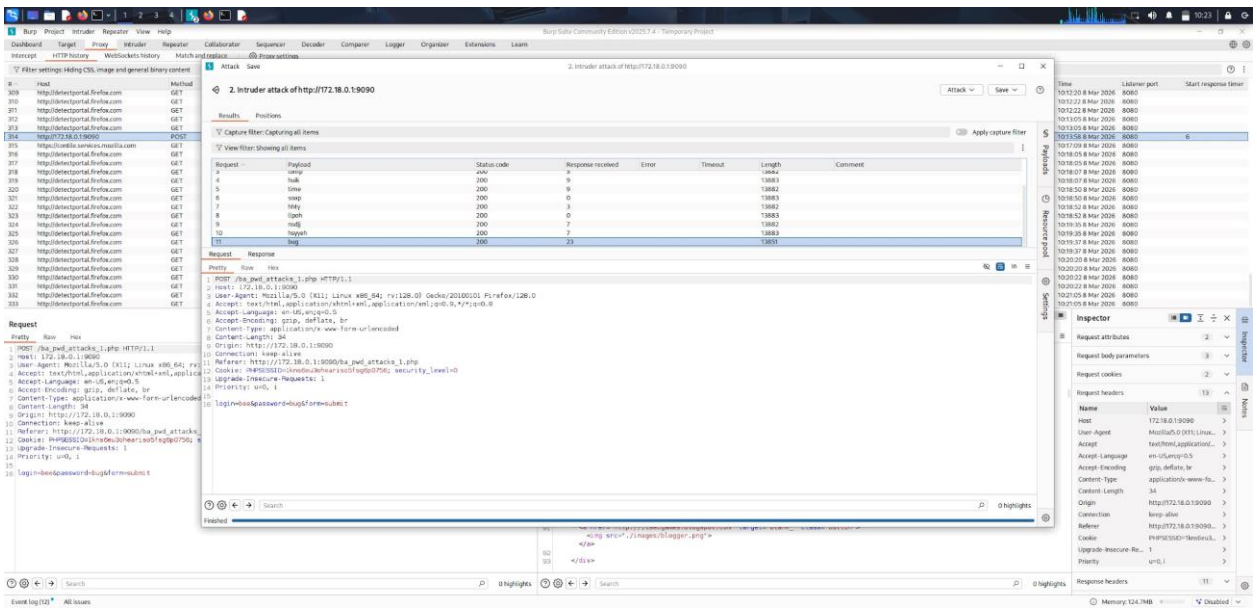

bWAPP - Session Manag
172.18.0.1:9090/smgmt_sessionid_url.php?PHPSESSID=gjv0dutvka2ptdlqmsk5bfqjg2
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Screenshot taken
View image
bWAPP
an extremely buggy web app !
Choose your bug
bWAPP v2.2
Hack
Set your security level:
low
Set
Current: low
Bugs Change Password Create User Set Security Level Reset Credits Blog Logout Welcome Tina
/ Session Mgmt. - Session ID in URL /
Session IDs should never be exposed in the URL!


bWAPP - Session Manag
172.18.0.1:9090/smgmt_sessionid_url.php?PHPSESSID=gjv0dutvka2ptdlqmsk5bfqjg2
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
bWAPP
an extremely buggy web app !
Choose your bug
bWAPP v2.2
Hack
Set your security level:
low
Set
Current: low
Bugs Change Password Create User Set Security Level Reset Credits Blog Logout Welcome Tina
/ Session Mgmt. - Session ID in URL /
Session IDs should never be exposed in the URL!


bWAPP - Broken Authen
Inbox (2,907) - ayomide
172.18.0.1:9090/ba_logout.php
bWAPP
an extremely buggy web app !
Choose your bug
Hack
Set your security level
Set
Current: low
Bugs
Change Password
Create User
Set Security Level
Reset
Credits
Blog
Logout
/ Broken Auth. - Logout Management /
Click here to logout.


Burp Suite Community Edition v2025.7.4 - Temporary Project
Request
Response
Inspector
GET /login.php HTTP/1.1
HTTP/1.1 200 OK
bWAPP - Login





Remediation Suggestions

To mitigate authentication-related vulnerabilities, the following measures are recommended:

- Implement account lockout mechanisms or rate limiting to prevent brute force password attacks.
- Store session identifiers securely within HTTP cookies rather than including them in URLs.
- Enforce HTTPS across all authentication-related pages to protect credentials during transmission.
- Ensure session tokens are invalidated immediately upon user logout.
- Implement secure session management practices such as session timeouts and session ID regeneration after authentication.

SQL INJECTION

Methodology

The application bWAPP was tested for SQL Injection vulnerabilities by submitting specially crafted SQL payloads into input fields that interact with the backend database, including search parameters and authentication forms. These inputs were intercepted and analyzed using Burp Suite in order to modify requests and observe how the server processed the injected data. Various SQL payloads were inserted into GET parameters, POST parameters, JSON inputs, and login form fields to determine whether the application properly validated or sanitized user-supplied input before executing database queries.

Vulnerabilities Identified

1. SQL Injection in GET Request Parameters

Testing of GET request parameters revealed that user input was directly incorporated into backend SQL queries without proper sanitization. When SQL payloads such as:

' OR 1=1 -- were submitted through the input field, the application returned SQL-related error messages indicating improper handling of database queries it also showed the type of database being used which is also a potential vulnerability.These responses confirmed that the application is vulnerable to SQL Injection attacks through GET parameters.

2. SQL Injection in POST Request Parameters

POST-based input fields were also tested for SQL Injection vulnerabilities. Similar SQL payloads were injected into POST request parameters, which resulted in database error messages being returned by the application also giving the type of database used which was MYSQL. These responses suggested that user input was being processed directly by the backend SQL query engine without adequate filtering or validation.

3. SQL Injection in JSON-Based Inputs

Testing was also conducted on JSON-based request parameters where input values were manipulated using SQL injection payloads. The application again returned SQL-related error responses, indicating that JSON input values were also being processed without proper sanitization before being incorporated into SQL queries. The error message is :

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '...' at line 1

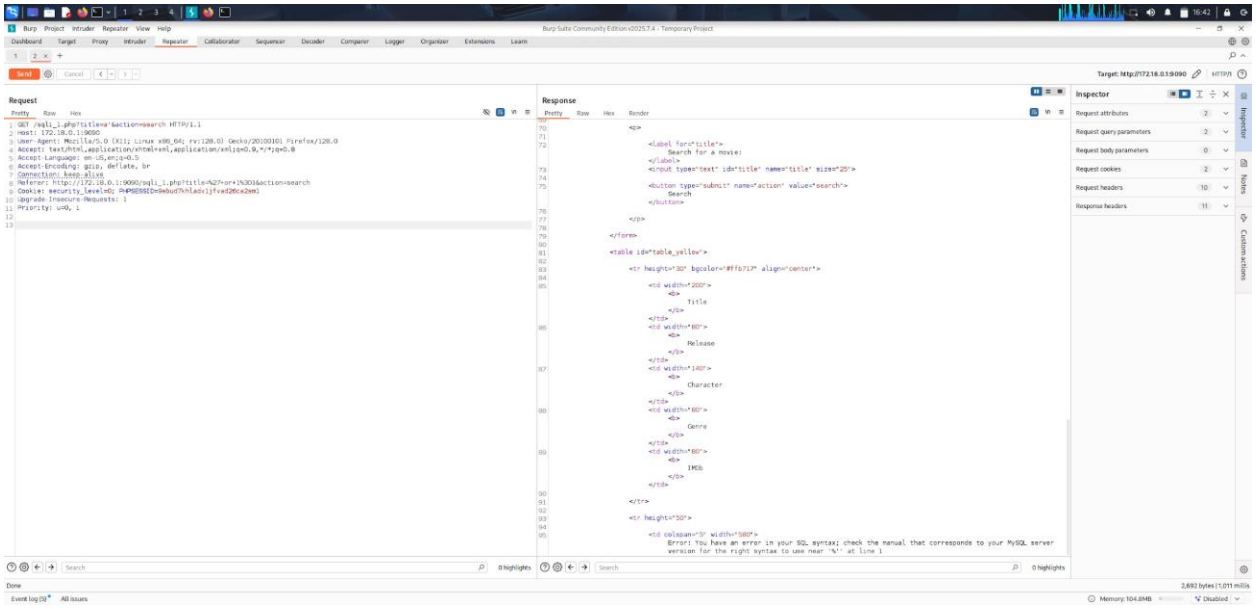
4. SQL Injection Authentication Bypass (Login Form)

During testing of the login functionality, it was discovered that authentication could be bypassed using SQL injection payloads. By injecting the following payload into the login form:

'--

the authentication query was altered, allowing access to the application without providing valid user credentials. This demonstrates a critical SQL Injection vulnerability that could allow attackers to bypass authentication controls.

Screenshots of Exploitation


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.7.4 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Send Cancel
Target: http://172.18.0.1:9090
Request
Pretty Raw Hex
1 GET /sqli_1.php?title=a'&action=search HTTP/1.1
2 Host: 172.18.0.1:9090
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://172.18.0.1:9090/sqli_1.php?title=%27+or+1%3D1&action=search
9 Cookie: security_level=0; PHPSESSID=9ebud7khladvljfvad2ttca2em1
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13
Response
Pretty Raw Hex Render
70
71
72 <label for="title">
Search for a movie:
</label>
73 <input type="text" id="title" name="title" size="25">
74
75 <button type="submit" name="action" value="search">
Search
</button>
76
77
78
79 </form>
80
81 <table id="table_yellow">
82
83 <tr height="30" bgcolor="#ffb717" align="center">
84
85 <td width="200">

Title

</td>
86 <td width="80">

Release

</td>
87 <td width="140">

Character

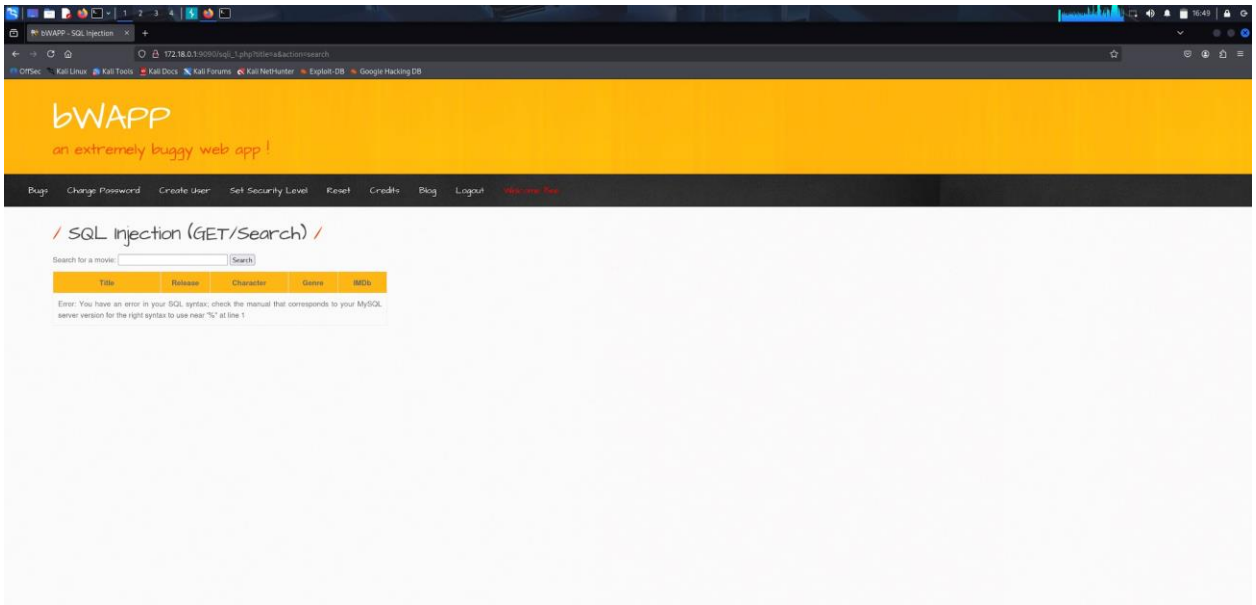
</td>
88 <td width="80">

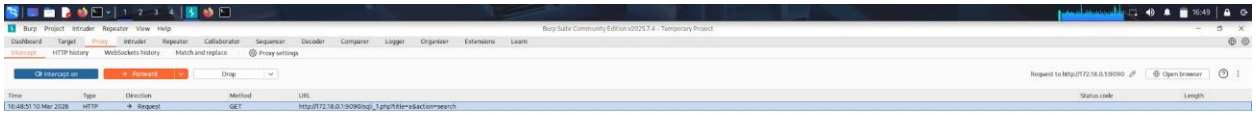
Genre

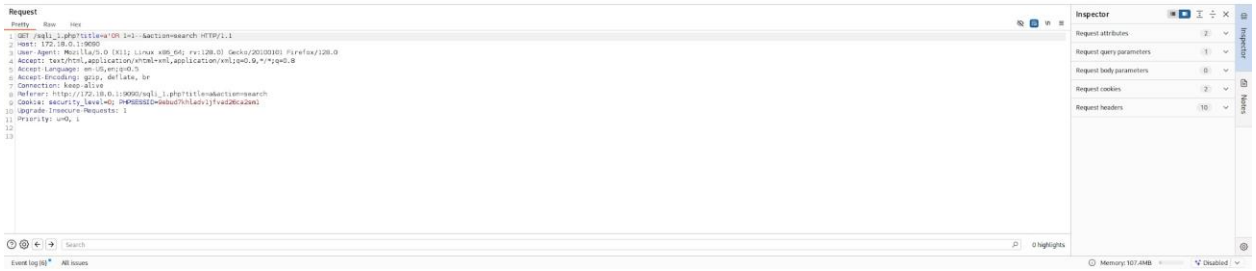
</td>
89 <td width="80">

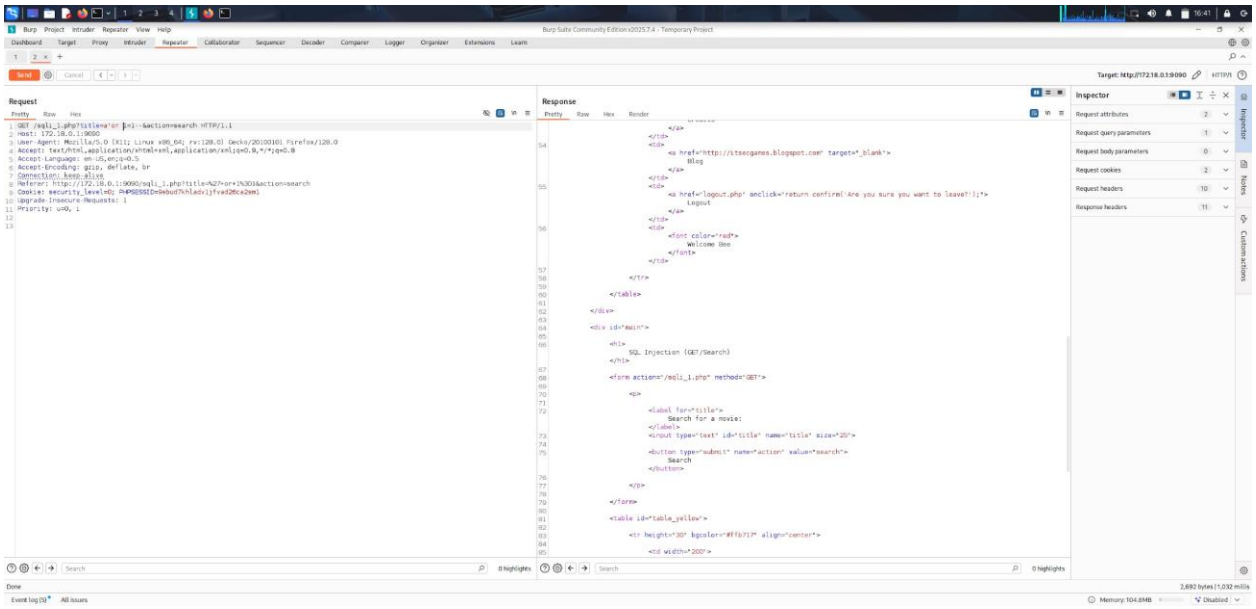
IMDb

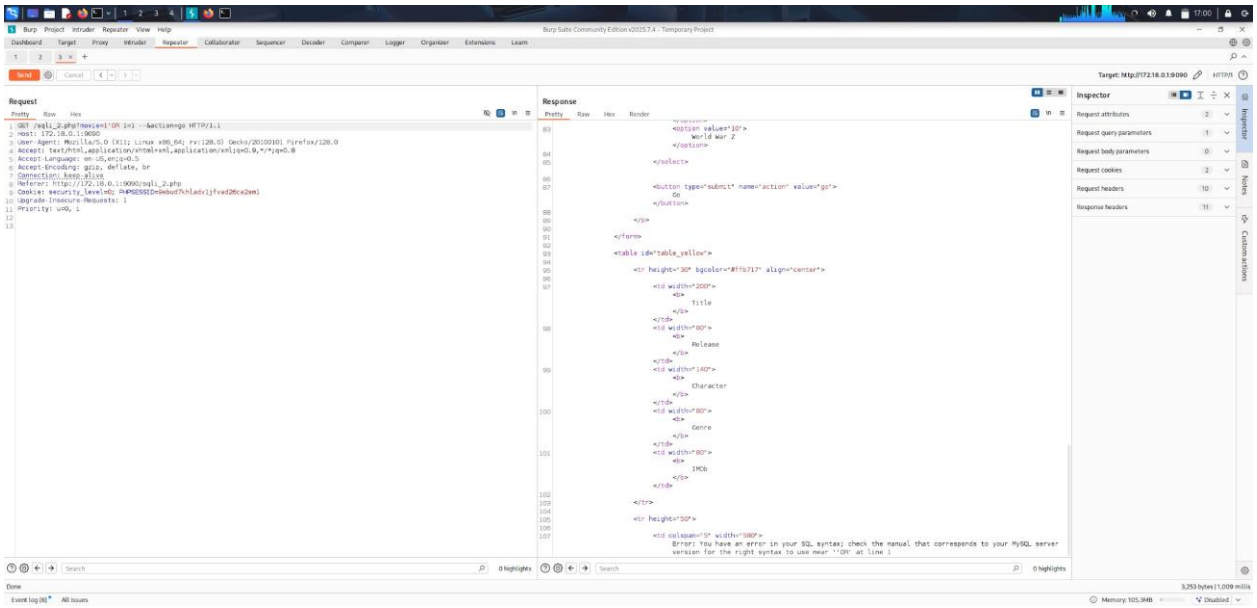
</td>
90
91 </tr>
92
93 <tr height="50">
94
95 <td colspan="5" width="580">
Error: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '%' at line 1
Inspector
Request attributes 2
Request query parameters 2
Request body parameters 0
Request cookies 2
Request headers 10
Response headers 11
Inspector
Notes
Customizations
Search 0 highlights
Search 0 highlights
Done
2,692 bytes | 1,011 millis
Event log (5)
All issues
Memory: 104.8MB
Disabled

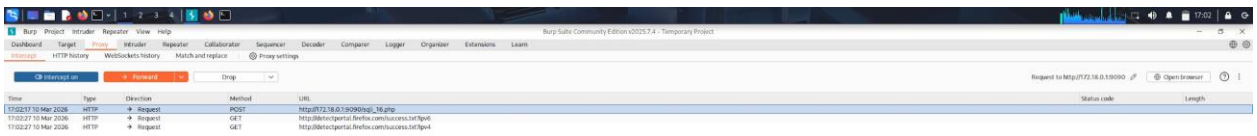

bWAPP - SQL Injection
172.18.0.1:9090/sqli_1.php?title=a&action=search
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
bWAPP
an extremely buggy web app !
Bugs Change Password Create User Set Security Level Reset Credits Blog Logout Welcome Tom
/ SQL Injection (GET/Search) /
Search for a movie: Search
Title Release Character Genre IMDb
Error: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '%' at line 1

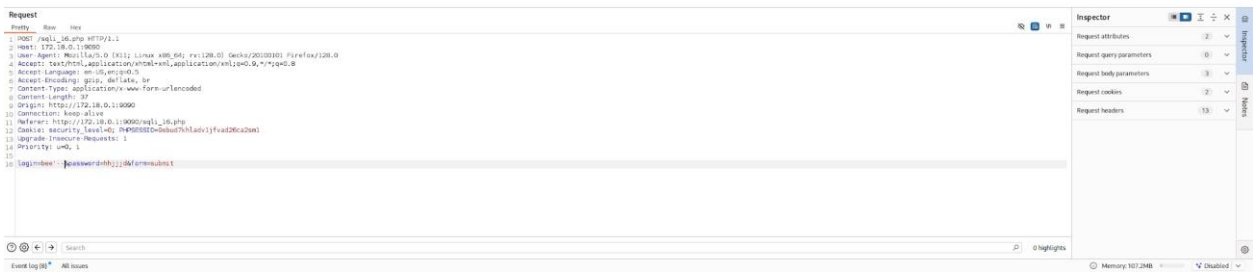

Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.7.4 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on Forward Drop
Request to http://172.18.0.1:9090
Open browser
Time Type Direction Method URL Status code Length
16:48:51 10 Mar 2026 HTTP → Request GET http://172.18.0.1:9090/sqli_1.php?title=a&action=search


Request
Pretty Raw Hex
GET /sqli_1.php?title=a'OR 1=1--&action=search HTTP/1.1
Host: 172.18.0.1:9090
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://172.18.0.1:9090/sqli_1.php?title=a&action=search
Cookie: security_level=0; PHPSESSID=9e8ud7khladvljfvad2dca2em1
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Inspector
Request attributes 2
Request query parameters 1
Request body parameters 0
Request cookies 2
Request headers 10
Notes
Search
0 highlights
Event log (6) All issues
Memory: 107.4MB
Disabled


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.7.4 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Send Cancel
Target: http://172.18.0.1:9090 HTTP/1
Request
Pretty Raw Hex
GET /sqli_1.php?title=a'or 1=1--&action=search HTTP/1.1
Host: 172.18.0.1:9090
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://172.18.0.1:9090/sqli_1.php?title=%27+or+1%3D1&action=search
Cookie: security_level=0; PHPSESSID=9e8ud7khladvljfvad2dca2em1
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Response
Pretty Raw Hex Render
54 <a href="http://itsecgames.blogspot.com/" target="_blank">
Blog
55 <a href="logout.php" onclick="return confirm('Are you sure you want to leave?');">
Logout
56 <font color="red">
Welcome Bee
57 </tr>
59 </table>
62
64
66 <h1>
SQL Injection (GET/Search)
67 </h1>
68 <form action="/sqli_1.php" method="GET">
70
72 <label for="title">
Search for a movie:
73 <input type="text" id="title" name="title" size="25">
75 <button type="submit" name="action" value="search">
Search
76
78 </form>
81 <table id="table_yellow">
82 <tr height="30" bgcolor="#ffb717" align="center">
84 <td width="200">
Inspector
Request attributes 2
Request query parameters 1
Request body parameters 0
Request cookies 2
Request headers 10
Response headers 11
Notes
Customizations
Search
0 highlights
Done
2,692 bytes | 1,032 millis
Event log (5) All issues
Memory: 104.8MB
Disabled







Remediation Suggestions

To prevent SQL Injection vulnerabilities, the following security measures are recommended:

- Implement parameterized queries (prepared statements) to separate SQL logic from user input.
- Perform strict input validation and sanitization on all user-supplied data.
- Avoid constructing SQL queries using dynamic string concatenation.
- Use ORM frameworks that automatically parameterize database queries.
- Deploy security monitoring tools capable of detecting SQL Injection attempts.
- Implement rules to input that doesn't allow certain characters linked to SQL code like ‘ ,--, 1=1,union

Cross-Site Scripting (XSS)

Methodology

The web application bWAPP was tested for Cross-Site Scripting vulnerabilities by injecting JavaScript payloads into input fields that reflect user-supplied data in the browser. These inputs were intercepted and analyzed using Burp Suite to observe how the application processed and rendered the supplied input.

Payloads containing JavaScript code were submitted through GET parameters, POST parameters, and JSON-based inputs to determine whether the application properly sanitized user input before displaying it in the browser.

Vulnerabilities Identified

1. **Reflected XSS via GET Parameters**

Testing of GET request parameters revealed that user input was reflected directly in the browser without proper sanitization. When the following payload was injected into the input field:

```
<script>alert(1)</script>
```

the browser executed the injected script and displayed a JavaScript alert message. This behavior confirms that the application reflects user input without filtering or encoding it, allowing attackers to execute arbitrary scripts in the victim's browser.

2. **Reflected XSS via POST Parameters**

POST request inputs were also tested for Cross-Site Scripting vulnerabilities. When the same payload:

```
<script>alert(1)</script>
```

was submitted through POST parameters, the application again rendered the script in the browser and triggered a JavaScript alert. This confirmed that POST inputs were not properly sanitized before being returned in the response.

3. **Reflected XSS in JSON-Based Inputs**

Testing was also conducted on JSON-based input parameters. Initially, the following payload was used:

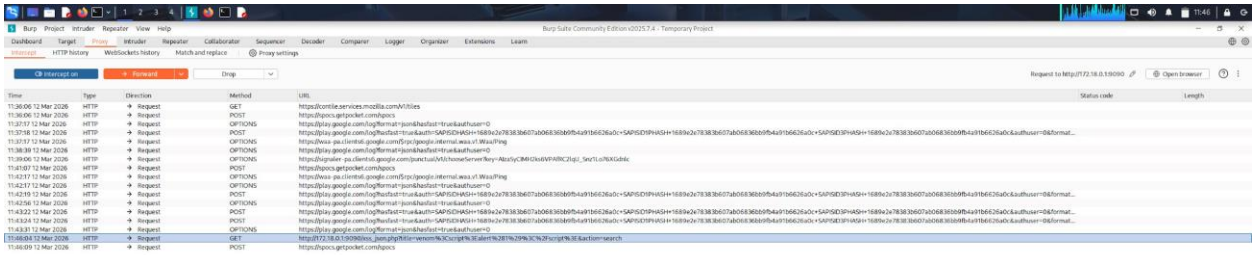
```
<script>alert(1)</script>
```

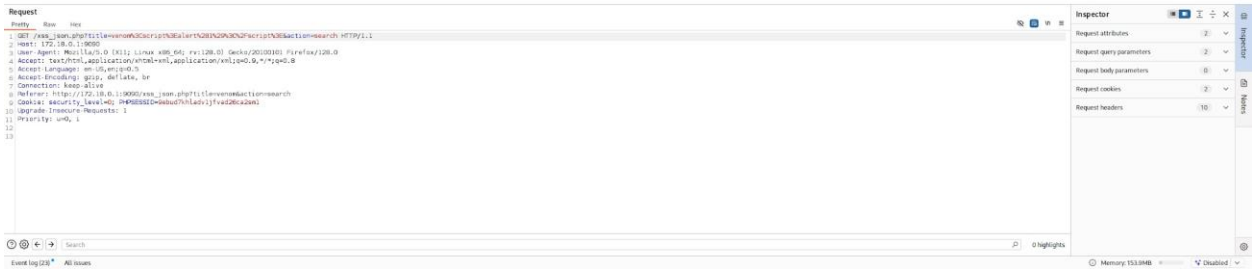
However, this payload resulted in an error message rather than executing within the browser, suggesting that the application was partially restricting script tags in that input context. Further testing was conducted using an alternative payload:

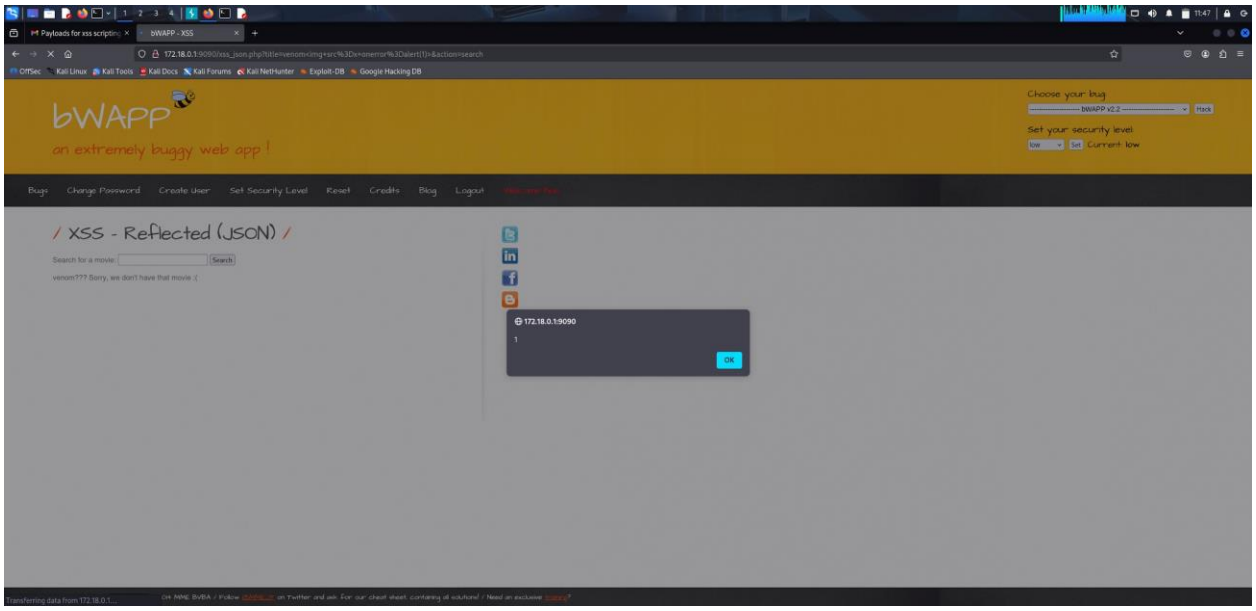
```
<img src=x onerror=alert(1)>
```

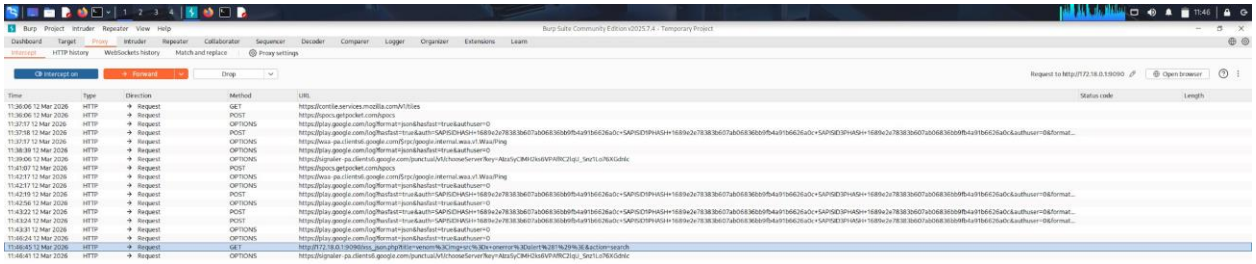
This payload successfully triggered a JavaScript alert when processed by the application, confirming the presence of a Cross-Site Scripting vulnerability within the JSON input handling mechanism.

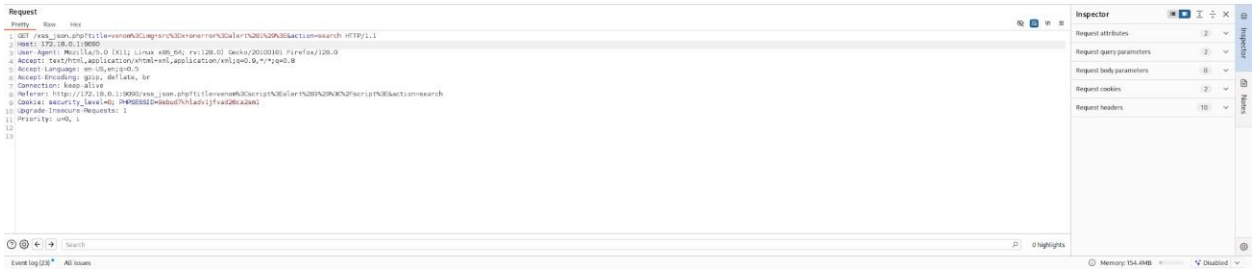
Screenshots of Exploitation

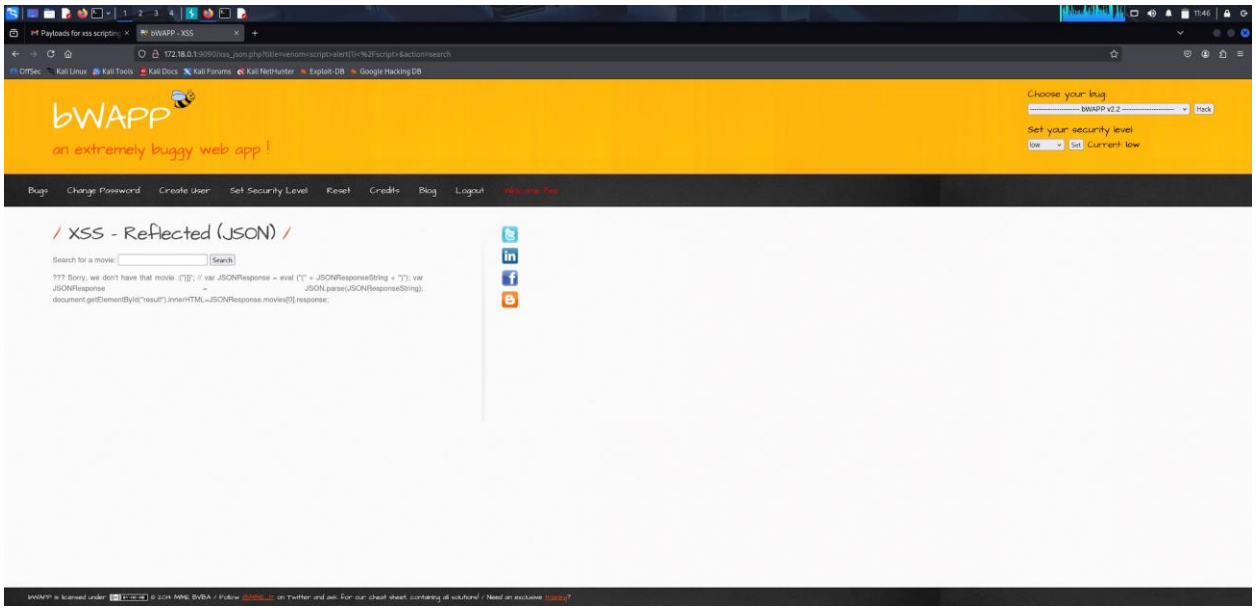

Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.7.4 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on Forward Drop
Request to http://172.18.0.1:9090
Open browser
Time Type Direction Method URL Status code Length
11:36:06 12 Mar 2026 HTTP Request GET https://content.services.mozilla.com/v1/tiles
11:36:06 12 Mar 2026 HTTP Request POST https://spocs.getpocket.com/spocs
11:37:17 12 Mar 2026 HTTP Request OPTIONS https://play.google.com/log?format=json&hasfast=true&authuser=0
11:37:18 12 Mar 2026 HTTP Request POST https://play.google.com/log?hasfast=true&auth=SAPISIDHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c+SAPISID1PHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c+SAPISID3PHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c&authuser=0&format...
11:37:17 12 Mar 2026 HTTP Request OPTIONS https://lwaa-pa.clients6.google.com/$rpc/google.internal.waa.v1.Waa/Ping
11:38:39 12 Mar 2026 HTTP Request OPTIONS https://play.google.com/log?format=json&hasfast=true&authuser=0
11:39:06 12 Mar 2026 HTTP Request OPTIONS https://signaler-pa.clients6.google.com/punctual/v1/chooseServer?key=AIzaSyCBMHGkx6VPARC2Igu_5nzTLo7NXGdnbc
11:41:07 12 Mar 2026 HTTP Request POST https://spocs.getpocket.com/spocs
11:42:17 12 Mar 2026 HTTP Request OPTIONS https://lwaa-pa.clients6.google.com/$rpc/google.internal.waa.v1.Waa/Ping
11:42:17 12 Mar 2026 HTTP Request OPTIONS https://play.google.com/log?format=json&hasfast=true&authuser=0
11:42:19 12 Mar 2026 HTTP Request POST https://play.google.com/log?hasfast=true&auth=SAPISIDHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c+SAPISID1PHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c+SAPISID3PHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c&authuser=0&format...
11:42:56 12 Mar 2026 HTTP Request OPTIONS https://play.google.com/log?format=json&hasfast=true&authuser=0
11:43:22 12 Mar 2026 HTTP Request POST https://play.google.com/log?hasfast=true&auth=SAPISIDHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c+SAPISID1PHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c+SAPISID3PHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c&authuser=0&format...
11:43:24 12 Mar 2026 HTTP Request POST https://play.google.com/log?hasfast=true&auth=SAPISIDHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c+SAPISID1PHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c+SAPISID3PHASH+1689e2e78383b607ab06836bb9fb4a91b6626a0c&authuser=0&format...
11:43:31 12 Mar 2026 HTTP Request OPTIONS https://play.google.com/log?format=json&hasfast=true&authuser=0
11:46:04 12 Mar 2026 HTTP Request GET http://172.18.0.1:9090/xss_json.php?title=venom%3Cscript%3Ealert%281%29%3C%2Fscript%3E&action=search
11:46:09 12 Mar 2026 HTTP Request POST https://spocs.getpocket.com/spocs

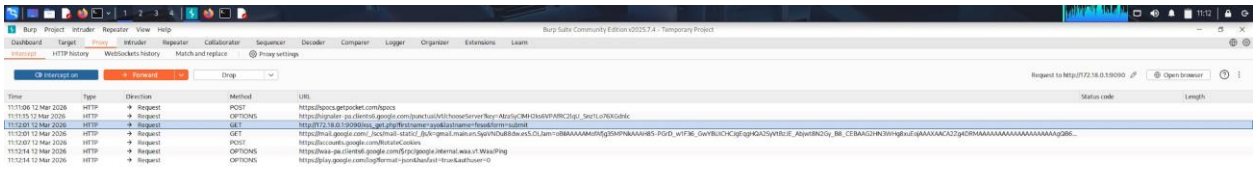

Request
Pretty Raw Hex
GET /xss_json.php?title=venom%3Cscript%3Ealert%281%29%3C%2Fscript%3E&action=search HTTP/1.1
Host: 172.18.0.1:9090
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://172.18.0.1:9090/xss_json.php?title=venom&action=search
Cookie: security_level=0; PHPSESSID=k6bu07k1advjjfvs2dcs2ah1
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Inspector
Request attributes 2
Request query parameters 2
Request body parameters 8
Request cookies 2
Request headers 10
Search
0 highlights
Event log (23)
All issues
Memory: 153.9MB
Disabled

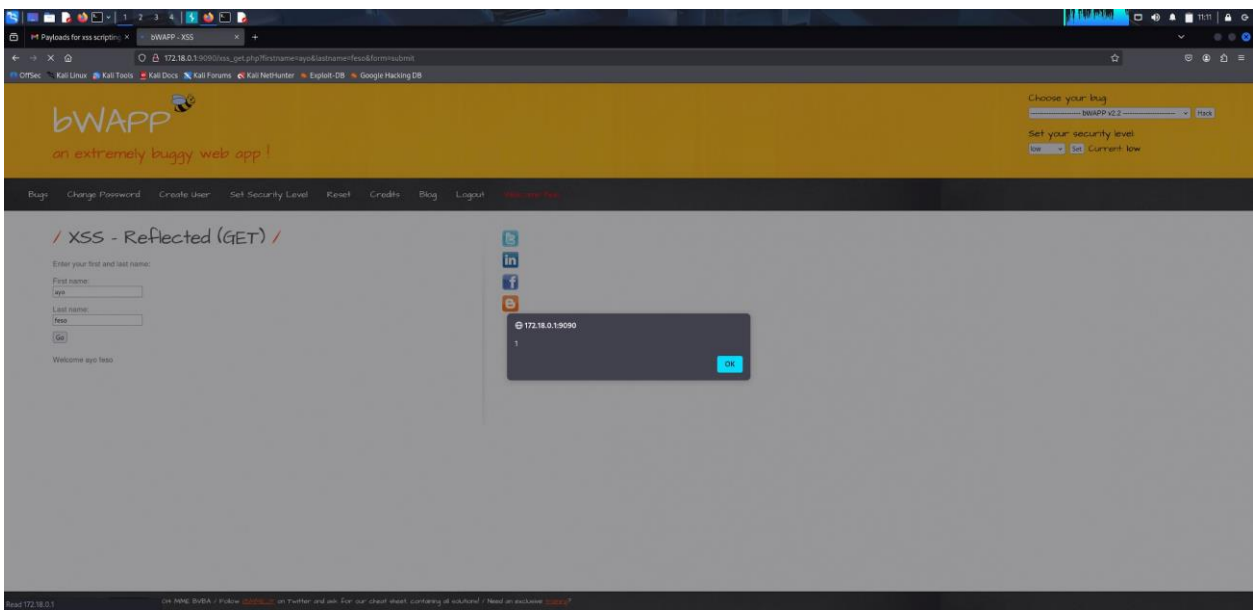

Payloads for xss scripting
bWAPP - XSS
172.18.0.1:9090/xss_json.php?title=venom<img+src%3Dx+onerror%3Dalert(1)>&action=search
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
bWAPP
an extremely buggy web app !
Choose your bug
bWAPP v2.2
Hack
Set your security level:
low
Set
Current: low
Bugs Change Password Create User Set Security Level Reset Credits Blog Logout
/ XSS - Reflected (JSON) /
Search for a movie:
Search
venom??? Sorry, we don't have that movie :(
172.18.0.1:9090
1
OK
Transferring data from 172.18.0.1...


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.7.4 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Intercept HTTP history WebSockets history Match and replace
Proxy settings
Intercept on
Forward
Drop
Request to http://172.18.0.1:9090
Open browser
Time Type Direction Method URL Status code Length


Request
Pretty Raw Hex
GET /xss_json.php?title=venom%3Cimg+src%3Dx+onerror%3Dalert%28%29%3E&action=search HTTP/1.1
Host: 172.18.0.1:9090
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://172.18.0.1:9090/xss_json.php?title=venom%3Cscript%3Ealert%281%29%3C%2Fscript%3E&action=search
Cookie: security_level=0; PHPSESSID=9ebud7kl4dvjjfvs2dcs2ah1
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Inspector
Request attributes 2
Request query parameters 2
Request body parameters 0
Request cookies 2
Request headers 10
Search
0 highlights
Event log (23)
All issues
Memory: 154.4MB
Disabled


Payloads for xss scripting
bWAPP - XSS
172.18.0.1:9090/xss_json.php?title=venom<script>alert(1)</%2Fscript>&action=search
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
bWAPP
an extremely buggy web app !
Choose your bug
bWAPP v2.2
Hack
Set your security level:
low
Set
Current: low
Bugs Change Password Create User Set Security Level Reset Credits Blog Logout
/ XSS - Reflected (JSON) /
Search for a movie:
Search
??? Sorry, we don't have that movie :(")]}'; // var JSONResponse = eval ("(" + JSONResponseString + ")"); var JSONResponse = JSON.parse(JSONResponseString); document.getElementById("result").innerHTML=JSONResponse.movies[0].response;
bWAPP is licensed under





Remediation Suggestions

- To mitigate Cross-Site Scripting vulnerabilities, the following measures are recommended:
- Implement proper input validation and output encoding for all user-supplied data.
- Sanitize input to prevent the execution of malicious scripts.
- Use Content Security Policy (CSP) to restrict the execution of unauthorized scripts.
- Encode user input before rendering it in HTML responses.
- Apply secure development practices that prevent untrusted data from being interpreted as executable code in the browser.

Command Injection

Methodology

The application bWAPP was tested for command injection vulnerabilities by submitting system command payloads into input fields that interact with server-side processes. These requests were

intercepted and analyzed using Burp Suite to determine whether the application properly validated or sanitized user-supplied input before executing server-side commands.
During testing, command injection payloads were inserted into the input parameter to observe whether the server would execute them as operating system commands. **Vulnerabilities identified:**

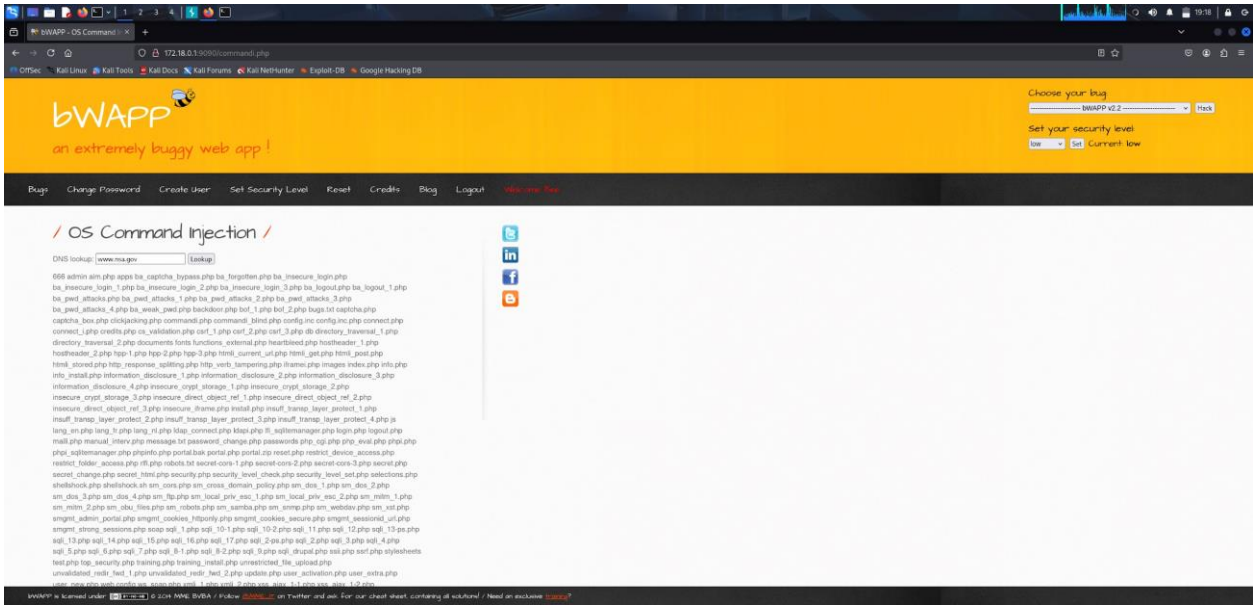
Command Injection via User Input:

Testing revealed that the application improperly handled user input and allowed operating system commands to be executed on the server.
The following payload was injected into the vulnerable parameter:

; ls

The ls command is a system command used to list files and directories on a Linux serveranf this command was used on the assumption that most servers are Linux based. After submitting the payload, the application returned a response displaying the list of files located on the server. This confirms that the application directly executed the injected command on the server without proper validation or sanitization of user input, demonstrating a command injection vulnerability.

Screenshots of Exploitation



Remediation Suggestions

To prevent command injection vulnerabilities, the following security measures are recommended:

- Validate and sanitize all user input before processing it on the server.
- Avoid executing system commands directly using user-supplied input.
- Use secure APIs or built-in functions instead of system command execution where possible.
- Implement strict allow-listing for permitted input values.
- Run applications with the least privilege principle to limit potential damage if exploitation occurs.

Insecure Direct Object Reference (IDOR)

Methodology

The application bWAPP was tested for insecure direct object reference vulnerabilities by analyzing how user identifiers and sensitive parameters were handled within HTTP requests. Requests were intercepted and modified using Burp Suite to determine whether users could manipulate parameters to access or modify resources belonging to other users.

Testing focused on parameters such as usernames, account identifiers, and transaction-related values to evaluate whether the server properly validated user authorization before processing requests.

Vulnerabilities Identified

1. **Unauthorized Modification of Another User's Secret**

During testing, it was discovered that the application allowed modification of user account data through request parameters. By intercepting the request responsible for changing a user's secret value, the username parameter was modified to reference another user account. After forwarding the modified request, the application processed the request successfully and updated the secret associated with the other user account. This indicates that the server did not verify whether the authenticated user was authorized to modify the target account.

2. **IDOR via XML Request Manipulation**

Further testing revealed another instance where user account information was transmitted using XML-formatted request data. The request contained both the username and the secret value in the XML body.

By intercepting the request and modifying the username field within the XML structure, it was possible to change the secret associated with another user account. This behavior was confirmed using Burp Suite Repeater, demonstrating that the application failed to enforce proper access control validation on the server side.

3. **Price Manipulation in Ticket Purchase Request**

Testing of the ticket purchasing functionality revealed that the price of the ticket was determined directly from the client-side request rather than being validated by the server.

By intercepting the purchase request and modifying the price parameter, the ticket price was successfully changed from $15 to $1. The server accepted the modified request and processed the transaction using the manipulated price value.

This indicates that the application relies on client-supplied values for critical transaction data without performing server-side validation.

Screenshots of Exploitation

bWAPP - Insecure DOR

172.17.0.1:9090/insecure_direct_object_ref_2.php

bWAPP

an extremely buggy web app !

Choose your bug: bWAPP v2.2 — Hack

Set your security level: low — Set — Current: low

Bugs | Change Password | Create User | Set Security Level | Reset | Credits | Blog | Logout | Welcome Bee

/ Insecure DOR (Order Tickets) /

How many movie tickets would you like to order? (15 EUR per ticket)

I would like to order 1 tickets.

Confirm

You ordered **1** movie tickets.

Total amount charged from your account automatically: **1 EUR**.

Thank you for your order!

bWAPP is licensed under CC BY NC NR © 2014 MME BVBA / Follow @MME_IT on Twitter and ask for our cheat sheet, containing all solutions! / Need an exclusive training?

Burp Suite Community Edition v2025.7.4 – Temporary Project

Screenshot taken

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response timer
190	http://172.17.0.1:9090	POST	/insecure_direct_object_ref_2.php	✔		200	13880	HTML	php	bWAPP - Insecure DOR			172.17.0.1		16:20:13 13 Mar 2026	8080	33

Request

```
POST /insecure_direct_object_ref_2.php HTTP/1.1
Host: 172.17.0.1:9090
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 46
Origin: http://172.17.0.1:9090
Connection: keep-alive
Referer: http://172.17.0.1:9090/insecure_direct_object_ref_2.php
Cookie: security_level=0; PHPSESSID=2ht5fs83h1ckjqogb11e3dbjq3
Upgrade-Insecure-Requests: 1
Priority: u=0, i

ticket_quantity=1&ticket_price=15&action=order
```

Response

```
HTTP/1.1 200 OK
Date: Fri, 13 Mar 2026 20:20:13 GMT
Server: Apache/2.4.7 (Ubuntu)
X-Powered-By: PHP/5.5.9-1ubuntu4.14
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 13892
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html

<!DOCTYPE html>
<html>

  <head>

    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">

    <!--<link rel="stylesheet" type="text/css" href="https://fonts.googleapis.com/css?family=Architects+Daughter">-->
    <link rel="stylesheet" type="text/css" href="stylesheets/stylesheet.css" media="screen" />
    <link rel="shortcut icon" href="images/favicon.ico" type="image/x-icon" />

    <!--<script src="//html5shiv.googlecode.com/svn/trunk/html5.js"></script>-->
    <script src="js/html5.js">
    </script>

    <title>
      bwAPP - Insecure DOR
    </title>

  </head>

  <body>

    <header>

      <h1>
        bWAPP
```

Inspector: Request attributes 2 | Request body parameters 3 | Request cookies 2 | Request headers 13 | Response headers 11

Event log (11) | All issues | Memory: 109.2MB | Disabled

Burp Project Intruder Repeater View Help

Burp Suite Community Edition v2025.7.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 2 3 4

Send Cancel

Target: http://172.17.0.1:9090 HTTP/1

Request

Pretty Raw Hex

```
1  POST /insecure_direct_object_ref_2.php HTTP/1.1
2  Host: 172.17.0.1:9090
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Content-Type: application/x-www-form-urlencoded
8  Content-Length: 45
9  Origin: http://172.17.0.1:9090
10 Connection: keep-alive
11 Referer: http://172.17.0.1:9090/insecure_direct_object_ref_2.php
12 Cookie: security_level=0; PHPSESSID=2ht9fs83h10klqmgb1le3objq3
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 ticket_quantity=1&ticket_price=1&action=order
```

Response

Pretty Raw Hex Render

```
66          <h1>
                Insecure DOR (Order Tickets)
            </h1>

67
68          <form action="/insecure_direct_object_ref_2.php" method="POST">
69
70              <p>
                    How many movie tickets would you like to order? (15 EUR per ticket)
                </p>
71
72              <p>
                    I would like to order <input type="text" name="ticket_quantity" value="1" size="2">
                    tickets.
                </p>
73
74              <input type="hidden" name="ticket_price" value="15">
75
76              <button type="submit" name="action" value="order">
                    Confirm
                </button>
77
78          </form>
79
80          <br />
81
82          <p>
                You ordered <b>
                        1
                </b>
                movie tickets.
            </p>
            <p>
                Total amount charged from your account automatically: <b>
                        1 EUR
                </b>
                .
            </p>
            <p>
                Thank you for your order!
            </p>
83      </div>
84
85      <div id="side">

86
87          <a href="http://twitter.com/MME_IT" target="blank_" class="button">
                <img src="./images/twitter.png">
            </a>
88          <a href="http://be.linkedin.com/in/malikmesellem" target="blank_" class="button">
                <img src="./images/linkedin.png">
            </a>
89          <a href="http://www.facebook.com/pages/MME-IT-Audits-Security/104153019664877" target="blank_" class="button">
```

Inspector

Request attributes	2
Request query parameters	0
Request body parameters	3
Request cookies	2
Request headers	13
Response headers	11

Inspector Notes Customactions

0 highlights 0 highlights

Done 13,879 bytes | 1,001 millis

Event log (11) All issues Memory: 109.5MB Disabled

Burp Project Intruder Repeater View Help

Burp Suite Community Edition v2025.7.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 2 3

Send Cancel

Target: http://172.17.0.1:9090 HTTP/1

Request

Pretty Raw Hex

```
1  POST /xxe_2.php HTTP/1.1
2  Host: 172.17.0.1:9090
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4  Accept: */*
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Content-Type: text/xml; charset=UTF-8
8  Content-Length: 65
9  Origin: http://172.17.0.1:9090
10 Connection: keep-alive
11 Referer: http://172.17.0.1:9090/insecure_direct_object_ref_3.php
12 Cookie: security_level=0; PHPSESSID=2ht9fs83h10klqmgb1le3objq3
13 Priority: u=0
14
15 <reset>
       <login>
           bug
16     </login>
       <secret>
17         no bugs
18     </secret>
19 </reset>
```

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Date: Fri, 13 Mar 2026 20:06:12 GMT
3  Server: Apache/2.4.7 (Ubuntu)
4  X-Powered-By: PHP/5.5.9-1ubuntu4.14
5  Expires: Thu, 19 Nov 1981 08:52:00 GMT
6  Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
7  Pragma: no-cache
8  Content-Length: 30
9  Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html
12
13
14 bug
15 's secret has been reset!
```

Inspector

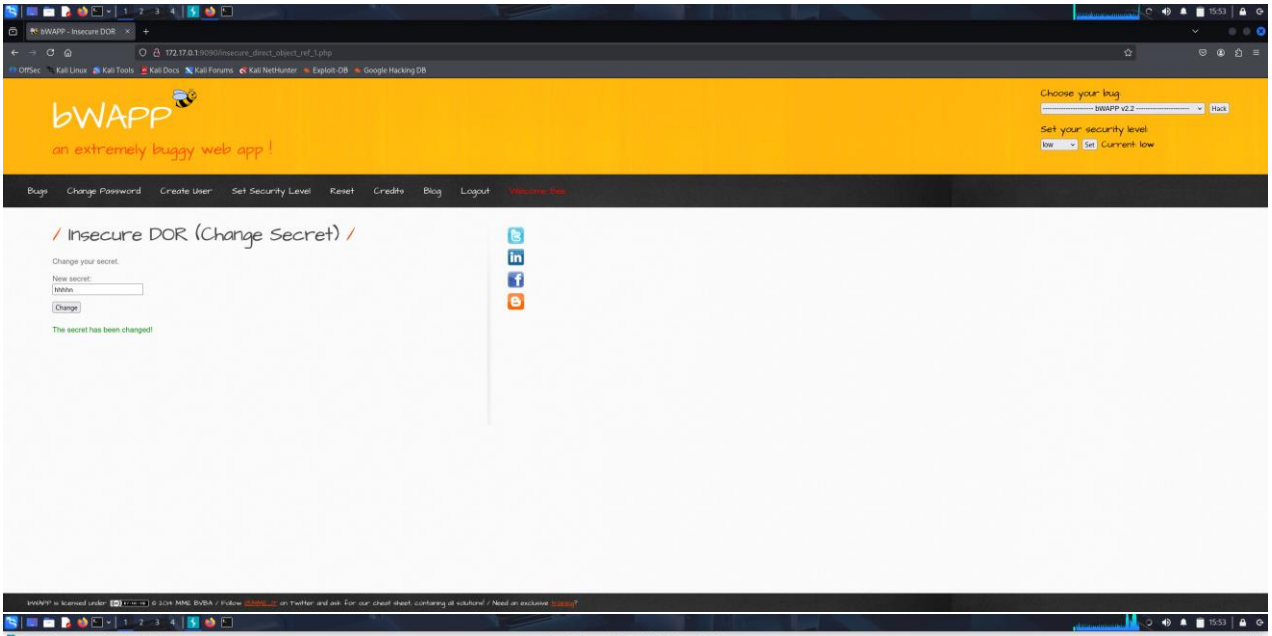
Request attributes	2
Request query parameters	0
Request cookies	2
Request headers	12
Response headers	10

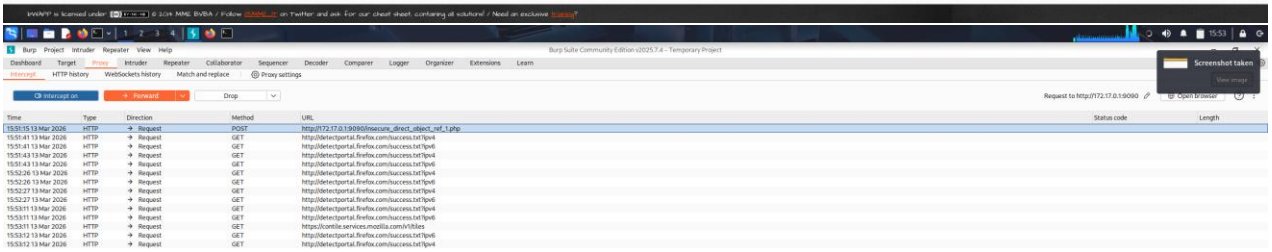
Inspector Notes Customactions

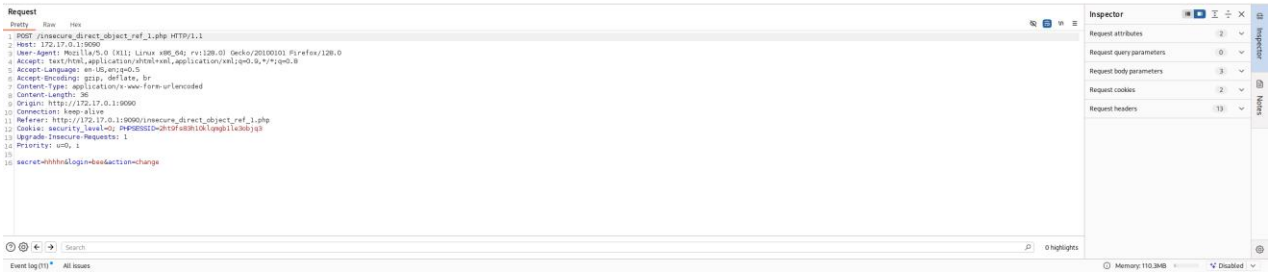
0 highlights 0 highlights

Done 392 bytes | 1,000 millis

Event log (11) All issues Memory: 109.2MB Disabled







Remediation Suggestions

To mitigate IDOR vulnerabilities, the following security measures are recommended:

Implement strict server-side authorization checks to ensure users can only access or modify resources they are permitted to control.

- Avoid exposing direct object references such as usernames or IDs without verifying access permissions.
- Use indirect reference mechanisms or secure object mapping to prevent unauthorized access.
- Validate all critical transaction parameters (such as pricing) on the server side rather than trusting client-supplied values.
- Implement comprehensive access control mechanisms to enforce proper authorization across all application functions.

The vulnerabilities identified during the assessment demonstrate several weaknesses in input validation, authentication mechanisms, and access control within the application. To provide a clear overview of these findings and their potential impact, a summary of the identified vulnerabilities and their associated risk levels is presented in the table below:

VULNERABILITIES	SEVERITY	NOTES/OBSERVATIONS
Identification & Authentication Failures	**Medium**	Weak login protection, session ID exposed in URL, and session not invalidated immediately after logout
SQL Injection	**Critical**	Database error messages exposed and login bypass achieved using SQL payloads
Cross-Site Scripting (XSS)	**Medium**	Reflected XSS via GET, POST, and JSON inputs allowing JavaScript execution in the browser
Command Injection	**Critical**	Server executed system command (ls) and returned directory listing
Insecure Direct Object Reference (IDOR) Medium	**Medium**	Manipulation of user identifiers allowed modification of another user's data and price tampering

Severity ratings were assigned based on the potential impact, exploitability, and security risk posed to the application. Vulnerabilities capable of leading to system-level compromise or unauthorized database access were categorized as Critical, whereas vulnerabilities primarily affecting user interactions or access control mechanisms were categorized as Medium.